Project Report

Heuristic Alpha Beta Checkers AI vs Human

4/24/2026

Solid Checkers

Chandler Guthrie, Matthew Covey, Jack Underhill, Jamil Staten

1. Project Overview

1.1. Introduction

To begin we decided on implementing a game for a player to play against an AI agent that has the capacity to create knowledgeable actions. For simplicity and the time constraints put into place we decided on Checkers as our game of choice. Checkers is somewhat close to Chess, however heavily simplified with its own difficult quirks. This made the project still challenging without it being too difficult to model and solve. We decided we would need an adversarial search algorithm so that we could identify both players in the search including identifying the features of the board to make educated actions. The Game would be implemented normally for a player vs player scenario with the AI in mind. We then chose to use a React Web interface for the user to interact with while using a python backend including for the AI algorithm. Utilizing techniques from the course and our own experiences with different frameworks we were able to complete all of these tasks. This left us with an AI agent comparable to an average player and a user interface that a person can use to play against it.

1.2. Research

1.2.1. Checkers Background

English Draught or American Checkers is a game board game consisting of a 8 by 8 board with light and dark sequential squares on a board. The board should be placed where a dark square is at the top right. The horizontal (rows) are formally known as **Ranks** and the vertical (columns) **Files.** The playable pieces are two colored checkers for opposing players red vs black and they can take two forms; a **Man** piece or a **King** piece. A Man is considered lower than the King such that it is unable to move freely on the board and is constrained to moving diagonally upward toward the opponents side of the board. Another advantage of a King is that it allows moving more than 1 space diagonally. Once a Man reaches the Rank closest to the opponent the Man is upgraded into a King hence forth granting moveability. A move toward the opponent is known as a Forward move and a move toward themself is a backward move.

To begin a game for a starting state of the board we must place the same amount of black and red checkers on each side of the board such that at least one rank is left empty individually for both players. The board can also number playable squares by counting the dark squares left to right on each rank downward starting from the black checkers for either player. The game always starts with the player who owns the black checkers.

To win a game a player must **Capture** all the opponents checkers by jumping over them diagonally. You are allowed to jump over consecutive checkers as long as a jump places the checker in another Capture position. Once a Player has gained an advantage it's known as a **Coup** or **Shot** by moving strategically around the board.

Since we applied Checkers these rules would apply differently if the game was the International Checkers version which can also be known as draughts from time to time. This board is 10 by 10 with slightly more sophisticated rules [1].

1.2.2. Checkers Rules

- A basic move made by a checker piece diagonally by one space in a diagonal unoccupied dark square. A Man can only move forward by one whereas a King can also move backward and any number of spaces.
- A Man is crowned a King once it lands itself at the rank closest to its opponent.
- A capture move done by a checker piece must be diagonal of an opponent's checker piece such that at least one space behind it is unoccupied for the piece to reside after capture. If the capturing piece is placed in another capture position the player has a choice to make another capture. (A Man piece must be directly behind it diagonally before capture while the King can be any number of playable squares behind it additionally it can end any playable squares over the piece after capture).
- During a turn if multiple playable checker pieces can capture an opponent's piece(s) the player is obligated to choose one of those pieces as an action.
- When a capturing piece ‘A’ is capturing other pieces it cannot capture a piece ‘B’ such that the piece ‘B’ is exactly next to a previously captured piece diagonally where the capturing piece would attempt to capture piece ‘B’ in the previously captured pieces’ direction.
- A Loss is incurred by a player by either losing all of their pieces or becoming unable to make any further moves with existing pieces.
- A Tie is a result of both players being unable to make any moves.

1.2.3. Algorithm

Noticing the many different ways that Computer Scientists have tackled this adversarial search problem we decided to use a simplified version of chinook [2]. Chinook contained a database of opening moves and used them with the AI and additionally other factors that made it more intelligent [2]. However we went ahead with not doing this for simplicity. Additionally there were other such solutions that existed such as the perfect proof which contained a greater database that eventually showed that a perfect playing game on both sides would result in a draw every time which seemed way out of scope for the semester [3]. To see more about how the AI algorithm works look at “3.3 AI Algorithm”.

1.3. Our Modeled Simplification

There are some modifications to the original game of checkers that we made to simplify the game for the project. First the board is represented as a 6 by 6 board. Second the king is not allowed to play multiple spaces diagonally which implies that the king can now only capture pieces directly behind another piece. This produces less states to deal with while also lessening the complexity of features.

2. How To Run The Project

2.1. Getting Started

Our project can be run either through the remotely hosted web version or by running the application locally. The hosted version is the easiest way to access the project because it requires no installation and allows the user to immediately interact with the game through a browser. For development, testing, or inspection of the codebase, the project can also be run locally.

The final system consists of two main parts: a React-based frontend and a Python backend. The frontend provides the graphical interface that allows the user to view the board and make moves, while the backend handles the game logic and AI decision-making. In addition to the web version, the repository also contains a CLI version of the game for local execution.

To run the project locally, the user should have Python 3.11 or newer installed for the backend, along with Node.js and npm for the frontend. On Windows, the repository includes batch scripts that simplify startup. On any platform, the frontend and backend can also be started manually.

2.2. Remote Hosted

The remotely hosted version of the project is the fastest way to run and demonstrate the system. To use it, the user simply opens the deployed frontend in a web browser and begins a game through the interface. The browser-based version connects the user interface to the hosted backend, which evaluates board states and returns AI moves during gameplay.

This version is recommended for general use, project demonstrations, and presentation purposes because it requires no local setup. It reflects the final direction of the project, which expanded from an earlier CLI-based idea into a GUI/web-based interface that allows a player to interact with the AI agent.

Remote hosted URL:
https://checkers-ai-agent.netlify.app/.

2.3. Self Hosted

To run the project locally, the frontend and backend must both be started. The backend should be started first, since it provides the API endpoints used by the frontend for game state updates and AI move generation. After the backend is running, the frontend can be started and opened in a browser for local play.

The project supports both a CLI version and a GUI version. For the GUI version, the Python backend runs in the “api/” portion of the project, while the React frontend runs in the “web/” portion. On Windows, the project includes helper batch scripts for launching either the full application stack or individual parts of it. For a manual cross-platform setup, the user installs the Python dependencies for the backend, starts the API server locally, then installs the frontend

dependencies and starts the React development server. By default, the frontend expects the local API to be available at “http://127.0.0.1:8000”, and the frontend then serves the application through its local development address.

A typical local setup process is:

1. Clone or download the repository.
2. Open the backend folder and install Python dependencies.
3. Start the Python backend server.
4. Open the frontend folder and install Node package dependencies.
5. Start the React development server.
6. Open the local frontend URL in a browser and play against the AI.

GitHub Repository URL:
https://github.com/CPTS-440-Artifical-Intelligence/Checkers_AI_Agent

3. Code Overview

3.1. Libraries, Tools, Languages

The project is split into three distinct layers, each using its own stack.

Backend (API Layer): Is written in Python, the backend is built on FastAPI served through Uvicorn. Redis is included as an optional dependency for game state persistence, while the in-memory fallback handles state with a thread-safe lock. The backend exposes a REST API with CORS middleware to allow the frontend to communicate across origins. Deployment is handled using Render.

Game Engine: Also written in Python, the engine is packaged separately as checkers-engine with no external dependencies beyond the standard library. It contains all game logic, including board state management, move generation, and the AI algorithm. It is consumed by the API layer as a local module.

Frontend: Built with React using JSX, bundled with Vite, and styled with Tailwind CSS. The frontend communicates with the backend through a lightweight API client and renders the board using layered components. It is deployed as a static site on Netlify.

CLI Version: A standalone Python script using only the standard library and dataclasses used for early development and local testing without the web interface.

Development Tools: npm and Node.js for the frontend, setuptools for Python packaging, ESLint for frontend linting, and batch scripts on Windows for launching the full stack locally.

3.2. Game Structure

3.2.1. Checkers Data Structure

The board is represented as a 6x6 list of lists of strings. Each cell holds one of five values “r” (red man), “R” (red king), “b” (black man), “B (black king), or “.” (empty). Only dark squares where (row + col) % 2 == 1 are never occupied. The game state is a dictionary carrying the board, whose turn it is, game status, winner, a must_capture flag, and the last move taken. The CLI version uses an integer encoded representation over 18 playable squares, with values like BLACK_MAN = 1, WHITE_MAN = -1.

3.2.2. State Generation

Legal move generation starts from the current board state and the active players' pieces. Each piece checks its valid diagonal directions. Men can only move forward, kings in all four directions. If any capture is available anywhere on the board, the must_capture flag is enforced and only capturing moves are returned. Multi-jump captures are handled by recursively checking whether the landing square places the piece in another capture position. Applying a move produces a new state by cloning the current board, moving the piece, removing capture pieces, and promoting men to kings when they reach the opposite black rank.

3.2.3. AI vs Player

The game is turn-based with the human playing as red and the AI playing as black (or vice versa, depending on configuration. Each turn, the frontend sends the current board state and the player's chosen move to the backend API. The API applies the move, then immediately invokes the AI engine to compute and apply the AI’s response move, returning the updated game state. This request/response cycle continues until a win or draw condition is detected, either a player loses all pieces or has no legal moves remaining.

3.3. AI Algorithm

3.3.1. Mini-Max / Alpha Beta Pruning / Heuristic

We decided to use an Adversarial Search algorithm known as Mini-Max, which is a Depth First Search implementation. This algorithm made it perfect to identify the most optimal paths for each player by identifying an evaluation function that looks at the player we are optimizing for, maximizing that evaluation value for the player, and minimizing the value for the opponent optimization, leading us to the best next state given they play optimally. We realize it's possible that a play won't play optimally and could actually lead to a win by the AI, possibly faster or just in general, but we did not include such a case.

However, to optimize the algorithm for speed, we chose to use two ways of doing so first is Alpha Beta Pruning which would prune out any known states and its entire branch on the basis

of knowing whether or not the Min or Max player would choose the state next given already branched out states by propagating the evaluation score of Min and Max during recursion given which one is being chosen. The second is creating a depth limit for how far we can search using a heuristic value, which is similar to the evaluation function. Instead of it being a value based on how well or badly the win or loss state was, it is an evaluation of how well it's currently doing in the middle of the game to optimize for a win. This means we had to ensure our heuristic was well-made, meaning it definitely needed to be admissible, otherwise situations like winning states wouldn't be preferred over non-winning states, keeping the game stagnant.

With the idea of the heuristic, we needed the features of the game itself to identify these states, so we decided to use a weighted linear function so that we could identify whether a game was either a loss or a win using a float from 0 to 1. This also made it easier to weigh the features, so we knew what would be considered a better feature for a given situation than another. The features we chose were simple, such as the number of checkers a man, or better yet, a King with a higher weight. How close a man was to becoming a King, so it could prioritize becoming a King over other moves at times. The most weighted feature was capturing an opponent's piece; without this, the winning states would not be taken seriously, and they would be overridden by non-winning states.

Now that we have identified our features, we need to represent a state and get the values for the features to be weighed against. So, to optimize for speed and space, we decided to use bits as our board. This allowed us to represent a state using only 5 variables or 5 x 64 bits of memory. While giving us the advantage of calculating the features using bitwise operations, which at times turned possible O(n) operations into O(1) operations compared to an array. Lastly, to prune out more states, we decided to create a similarity score for each state, where states that were deemed similar to another would not be counted in the search if the other was already among the chosen states.

3.3.2. Analytics

Let's look at some stats of how our agent performed in different situations of the game. We tested this by looking at 3 different state scenarios: the start state, a possible mid-game state, and lastly an end-game state. We ran the algorithm on different depth counts, 5 to 25, in increments of 5 on each of these states. We did this from the Black Checker owner position and the Red Checker owner position.

Looking at the Figures below, we realized that an increase of nodes does increase the time over 5-20 by a small increase from milliseconds to 2 seconds; however, reaching a depth of 25 mid-game, it would reach times of 16 seconds! We have theories of the problem, which will be discussed in the next section, "Problems & Solutions". Another noticeable analytical perspective is the heuristic values; it seemed that the AI was more practical to use at a higher depth from the start state. However, once we reached mid-game or end-game states,

improvements weren't as noticeable; in fact, this allowed us to choose to use 15 depth keeping search times within a realistic time span of roughly 1~5 seconds.

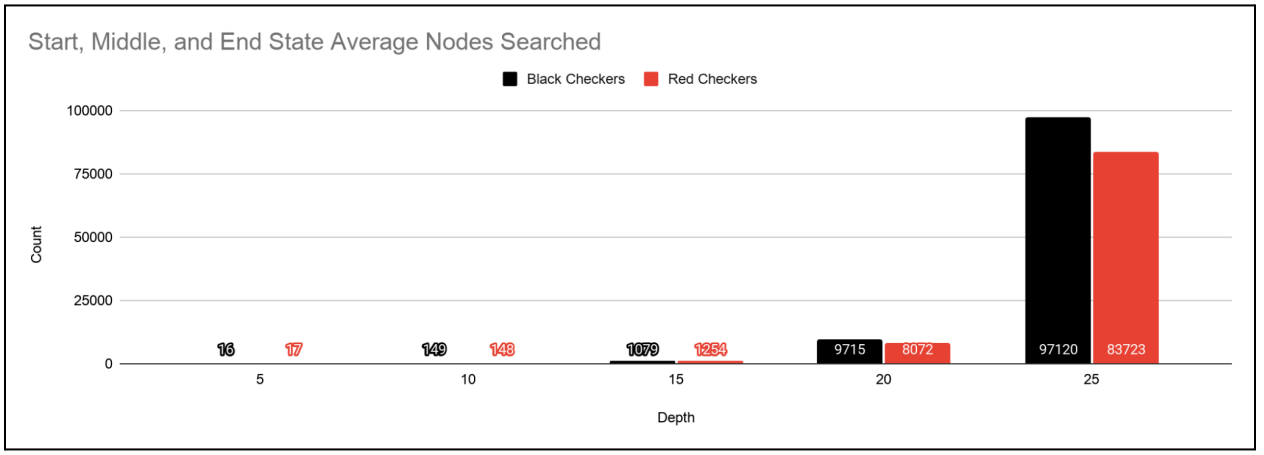

Figure 1 - 3.3.2

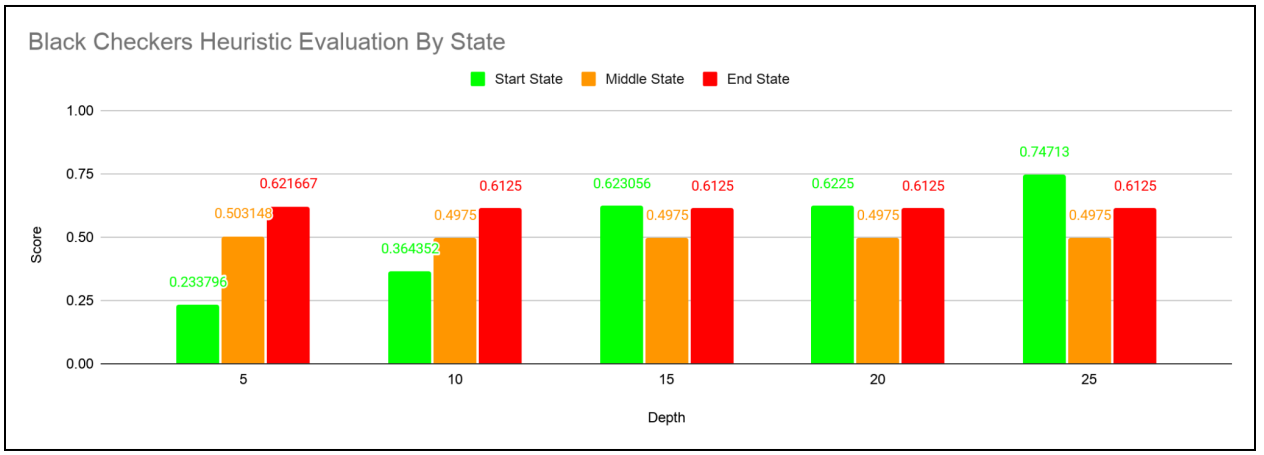

Figure 2 - 3.3.2

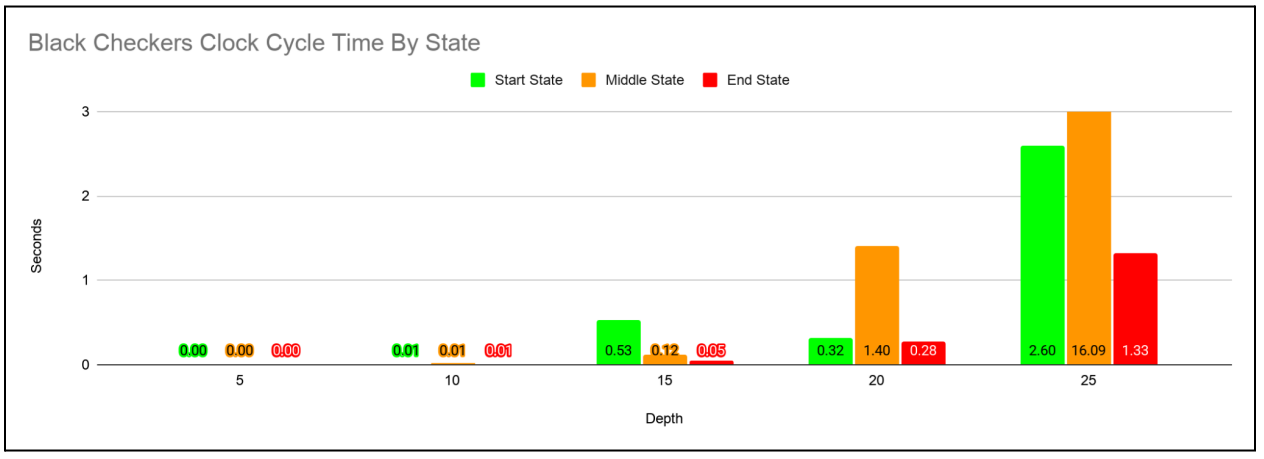

Figure 3 - 3.3.2

3.3.3. Problems & Solutions

We did run into some problems with the AI algorithm and came up with theories on such issues while presenting possible solutions to those issues. First, we realized we could use better features, as it's possible that at the endgame, the AI may go back and forth, not attempting a capture or an approach at doing so. To solve at least the issues of not attempting to do so, we would weight a feature of the distance between the black and red checkers such that a win state is when they are closer to each other. A major oversight was the mapping cost. We theorize that the time of mapping the array of the game state into the bits was causing this massive spike in time taken, since it would need to map back and forth during the algorithm. A simple fix to this would be to stick to one state representation, where the more optimal version is the bits representation. We assume that doing this would lower the overall time during the increase in nodes during search.

3.4. User Interface

The user interface of our project was designed to make interactions with the AI agent simple and intuitive. We implemented a React-based web interface that allows the player to view the checkerboard, select and move pieces, and immediately see the AI's responses after each turn. The interface serves as the visual layer of the system, while the Python backend handles the underlying game rules and decision-making logic. This UI direction also reflects the project's evolution from an earlier CLI concept into a browser-based application that is easier to demonstrate, test, and use interactively.

4. Results

4.1. Execution

The final product runs successfully for both terminal and web application, the latter with the React frontend deployed on Netlify and Python acting as the backend host on Render. People can click on the URL and immediately begin playing against the AI opponent without the need for installing any addons. The game will enforce all implemented rules for capturing, promotions, and alternating between turns. The AI responds within a realistic timeframe, making the game feel natural and engaging for people.

4.2. Evaluation

The AI agent demonstrated its worth by competing with people during testing and presentation. Search depth was decided at 15 since it is more balanced between quality and time to respond, allowing the agent to be both competitive and reflexive. The heuristic function flawlessly guided the AI toward favorable board states, prioritizing to capture, and king promotions. Although there was some hesitation displayed for some scenarios, the AI opponent performed consistently and intelligently throughout the game.

5. Final Thoughts

The project gave us a good hands-on experience for developing a system with an integrated AI agent. All major components were successfully integrated such as the 6x6 checkers engine, the heuristic minimax agent with alpha/beta pruning, and a web interface. The analytics that were collected helped us make informed decisions about search depth and bottleneck performance issues, specifically that surrounding state representation mapping between array and bitboard formats. If more time was dedicated to the project, these bottlenecks could be addressed by doing a single bitboard representation instead, or refining the heuristic function to improve the later game aggression, or doing the full 8x8 traditional checkers board. In the end, this was an amazing challenge that allowed us to show off our capabilities in developing a cohesive system.

References

[1] “International draughts,” Wikipedia, May 23, 2022.
https://en.wikipedia.org/wiki/International_draughts
[2] Chinook Agent:
https://en.wikipedia.org/wiki/Chinook_(computer_program)
[3] Perfect Play Proof of Agent:
https://www.science.org/cms/asset/7f2147df-b2f1-4748-9e98-1ac3afccfb76/pap.pdf